# Clase 15 · Diagrama de Estados

Bloque UML Análisis · Modelación y Diseño de Sistemas 2026 · UNC-ESCMB-AUSI

# Recap del bloque UML Análisis

- Clase 11 · Diagrama de Contexto
- Clase 12 · Casos de Uso
- Clase 13 · Modelo de Dominio
- Clase 14 · SSD
- Clase 15 · Diagrama de Estados
- Clase 16 · Validación del Triángulo de Larman (cierre)

# ¿Qué es un Diagrama de Estados?

Modela el CICLO DE VIDA de una clase / objeto.

Definición OMG UML 2.5.1: una máquina de estados describe el comportamiento de un objeto como una secuencia finita de estados y las transiciones entre ellos, desencadenadas por eventos.

- Estado · situación en la que el objeto espera un evento o realiza una actividad
- Evento · ocurrencia significativa que puede disparar una transición
- Transición · cambio de un estado a otro, con posible guarda y acción

Regla mental: pensá al objeto como una persona — ¿qué está haciendo ahora? ¿qué le puede pasar?

# Propósito del diagrama de estados

- Capturar ciclo de vida · cómo evoluciona una entidad desde su creación
- Modelar reglas de negocio temporales · qué transiciones son válidas y cuáles no
- Detectar estados olvidados · al dibujarlo aparecen estados que el texto no explicitó
- Guiar el diseño de tests · cada transición = un caso de prueba
- Input para validaciones · las guardas se convierten en pre-condiciones en el código

# ¿Cuándo dibujarlo?

SÍ vale la pena

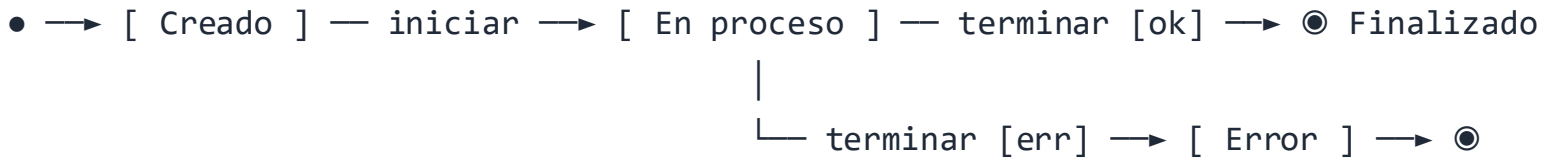
- ✓ La entidad atraviesa varios estados claramente diferenciados
- ✓ Hay reglas de negocio que dependen del estado actual
- ✓ Algunas transiciones están prohibidas (no toda transición es legal)
- ✓ Ciclo de vida largo o auditable (pedidos, turnos, ventas, cuentas)
- ✓ Querés derivar casos de prueba basados en estados

NO es necesario

- ✗ La entidad es pasiva (sólo datos, no evoluciona)
- ✗ Son 2 estados triviales (activo / inactivo)
- ✗ No hay reglas de transición — todo es permitido
- ✗ El flujo pertenece al sistema, no a UNA entidad (usá Actividades)

# Notación · vista global

Un diagrama de estados típico tiene:




- ● estado inicial (círculo negro relleno)
- [ … ] estado (rectángulo redondeado)
- ◉ estado final (círculo dentro de círculo)
- –► transición (flecha con evento [guarda] / acción)

# Metodología · identificar estados y transiciones

- Elegir la clase con ciclo de vida relevante · típicamente transaccionales (Pedido, Turno, Venta)
- Listar los estados observables del negocio · consultar con stakeholders. PascalCase, participio
- Identificar eventos que disparan cambios · operaciones del dominio: confirmar, pagar, cancelar, vencer
- Construir la matriz de transiciones · para cada par (estadoOrigen, evento) ¿a qué estado se transita?
- Añadir guardas y acciones donde haga falta · reglas de negocio condicionales
- Dibujar inicio y estados finales · un solo estado inicial; cero, uno o varios finales

# Buenas prácticas

- Un diagrama por clase con ciclo propio · no mezclar estados de varias entidades
- Nombres en participio / gerundio · EnProceso, Pagado, Cancelado — nunca Procesa
- Guardas explícitas cuando aporten claridad · [monto > 0], [stock ≥ cant]
- Evitar cadenas lineales triviales · si es A→B→C sin guardas ni eventos, no aporta nada
- Usar estados compuestos para reducir complejidad · si tenés 15 estados planos, agrupá en 3-4 compuestos
- Incluir el estado inicial siempre · obligatorio. El final solo si la entidad puede "morir"

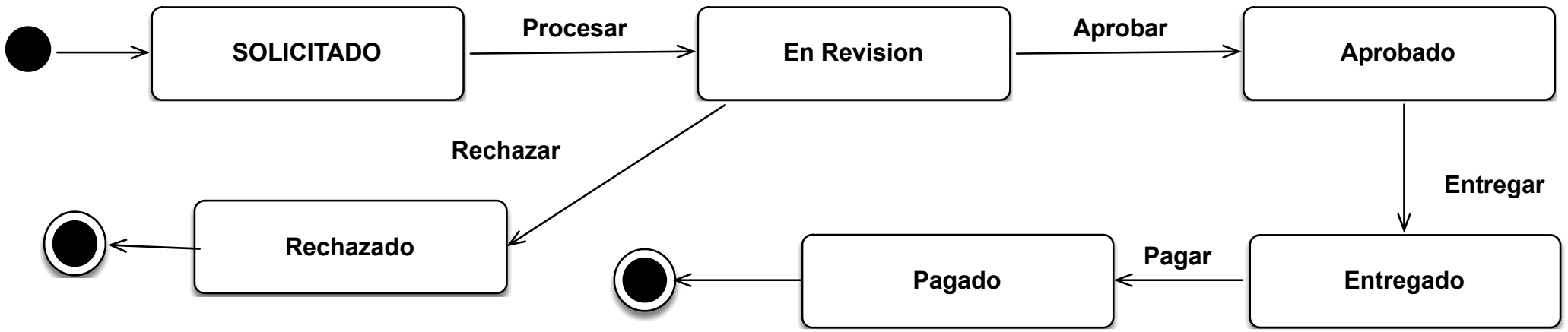
# Errores frecuentes

- ❌ Confundir estado con actividad · Procesando es estado; Procesar pago es actividad
- ❌ Muchos estados planos · más de 10-12 estados en el mismo nivel → usar compuestos
- ❌ Olvidar estado final · si la entidad puede morir, mostrarlo
- ❌ Transiciones sin evento · salvo desde el inicial, siempre debe haber un evento
- ❌ Mezclar control de flujo en el diagrama · el diagrama es del objeto, no del flujo del sistema
- ❌ Guardas incompatibles (overlap) · dos transiciones con guardas que pueden cumplirse a la vez

# Que busca representar

- Es una manera de caracterizar un cambio en el sistema, es decir que los objetos que lo componen modificaron su estado como respuesta a un suceso y/o por el paso del tiempo.

**Ej: Solicitud de un Préstamo**



# Aplicaciones

- Son útiles para modelar la vida de un objeto

- Un diagrama de estados muestra el flujo de control entre estados (en qué estados posibles puede estar “algo” y como se producen los cambios entre dichos estados)

# Elementos que lo componen




**Estado**: Un **estado** es una **condición o situación** en la vida de un objeto durante la cual **satisface una condición, realiza alguna actividad o espera algún evento**



**Transición**: Una **transición** es una **relación** entre dos estados que indica que un objeto que esté en el **primer estado** realizará ciertas **acciones** y **entrará** en el **segundo estado** cuando ocurra un **evento** especificado y se satisfagan unas **condiciones** especificadas

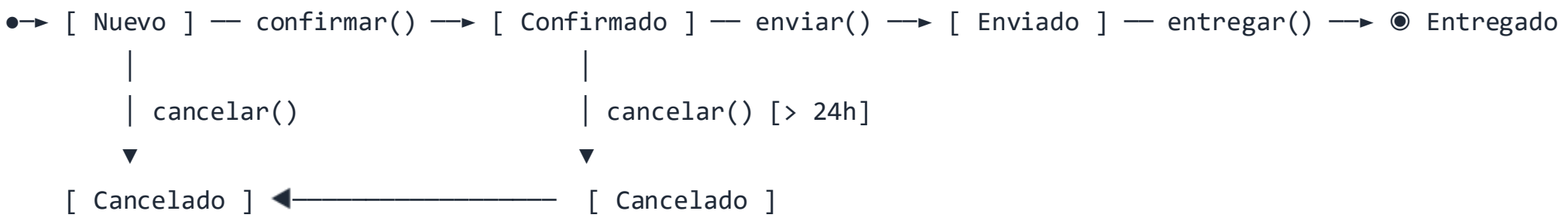


**Evento**: Un **evento** es la especificación de un **acontecimiento significativo** que ocupa un lugar en el **tiempo** y en el **espacio**. Es la aparición de un estímulo que puede (o no) activar una transición de estado



**Punto Final:**

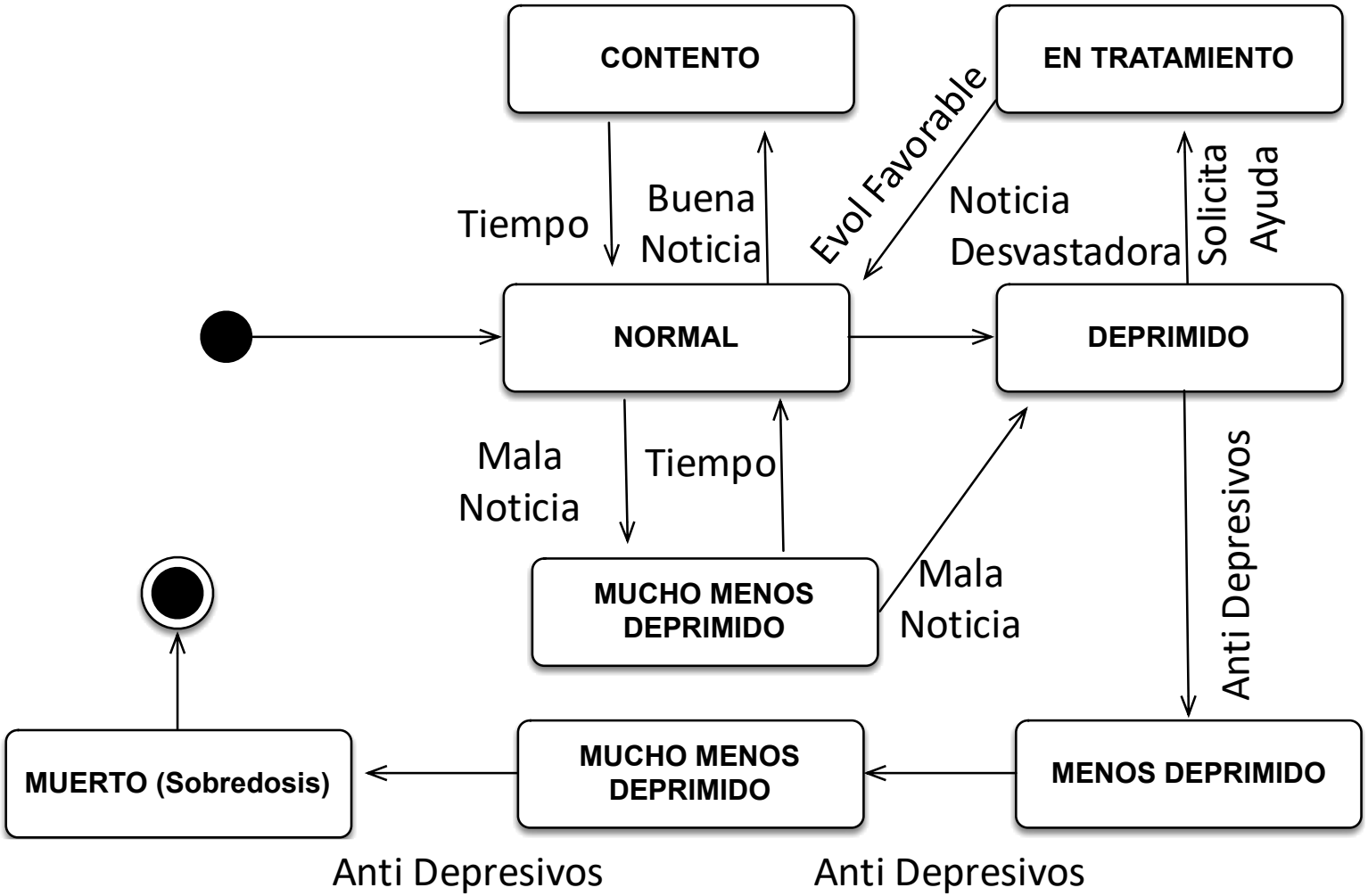
# Ejemplo 1 · Ciclo de vida de un Pedido




Reglas implícitas:

- No se puede enviar sin antes confirmar
- Solo se puede cancelar confirmado si pasaron menos de 24h
- Entregado y Cancelado son finales

# Ejemplo


CONTENTO
EN TRATAMIENTO
Tiempo
Buena Noticia
Evol Favorable
Noticia Desvastadora
Solicita Ayuda
NORMAL
DEPRIMIDO
Mala Noticia
Tiempo
Anti Depresivos
MUCHO MENOS DEPRIMIDO
Mala Noticia
MUERTO (Sobredosis)
MUCHO MENOS DEPRIMIDO
MENOS DEPRIMIDO
Anti Depresivos
Anti Depresivos

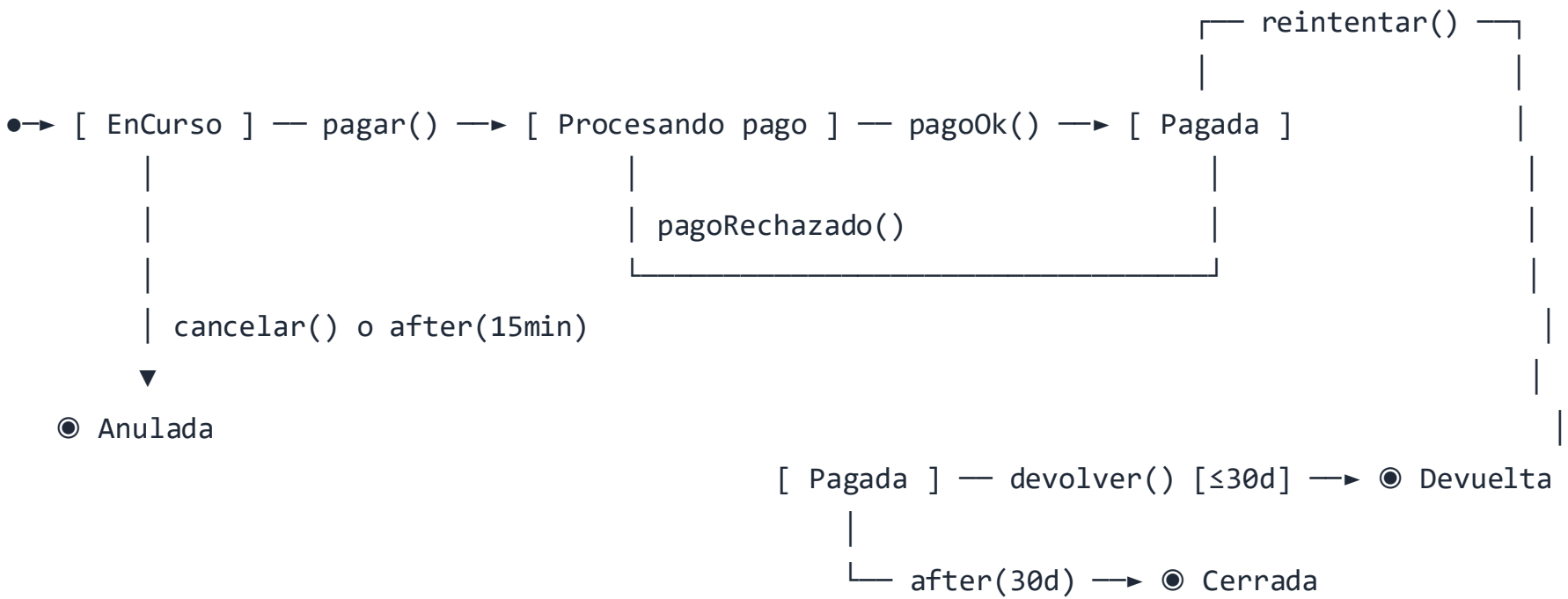
# Caso ChangoMás · ciclo de vida de una Venta

Entidad clave: Venta en el sistema de caja de autoservicio.

Reglas del negocio:

- Al iniciar la compra, la venta queda EnCurso
- Si el cliente abandona o vence un timeout, pasa a Anulada
- Cuando el cliente finaliza y paga, pasa a Pagada
- Si el pago falla en la pasarela, vuelve a EnCurso (puede reintentar)
- Si la pasarela autoriza, queda Pagada
- Una venta Pagada puede Devolverse dentro de 30 días → Devuelta

# ChangoMás · diagrama de estados de Venta



# ChangoMás · tabla de transiciones

| Estado origen | Evento | Guarda | Acción | Estado destino |
|---|---|---|---|---|
| (inicial) | iniciarCompra() | — | crear venta | EnCurso |
| EnCurso | escanearProducto(p) | — | agregar línea | EnCurso |
| EnCurso | pagar(medio) | total > 0 | enviar a pasarela | ProcesandoPago |
| EnCurso | cancelar() | — | log incidente | Anulada |
| EnCurso | after(15min) | — | autoCancelar | Anulada |
| ProcesandoPago | pagoOk() | — | imprimir ticket | Pagada |
| ProcesandoPago | pagoRechazado() | — | notificar | EnCurso |
| Pagada | devolver() | ≤ 30 días | reintegrar dinero | Devuelta |
| Pagada | after(30d) | — | archivar | Cerrada |

# ChangoMás · estados con entry/do/exit

```
[ EnCurso ]
   entry / asignarCajaLibre()
   do    / mostrarSubtotal()
   exit  / liberarCajaSiNoFinalizó()

[ ProcesandoPago ]
   entry / mostrarSpinner()
   do    / esperarPasarela()
   exit  / ocultarSpinner()

[ Anulada ]
   entry / registrarIncidente()
   entry / liberarCaja()
```

Las acciones de entry / exit son lo que el sistema realmente hace cuando cambia de estado.

# Trazabilidad SSD ↔ Estados ↔ UC

Cada operación del SSD corresponde a un evento del diagrama de estados. Si no coincide, falta uno de los dos.

| UC | Operación SSD | Evento de Estados | Transición |
|---|---|---|---|
| Realizar pago | iniciarCompra() | iniciarCompra | (inicial) → EnCurso |
| Realizar pago | escanearProducto(c) | escanearProducto | EnCurso → EnCurso (auto) |
| Realizar pago | pagar(medio) | pagar | EnCurso → ProcesandoPago |
| (interno pasarela) | — | pagoOk | ProcesandoPago → Pagada |
| Anular venta | cancelar() | cancelar | EnCurso → Anulada |

# Ejercicio en clase · 20 minutos

Consigna:

Sobre el caso propio (OAS):

- Identificá una entidad clave con ciclo de vida (Pedido, Turno, Solicitud, Trámite, etc.)
- Listá al menos 5 estados observables del negocio
- Para cada estado indicá los eventos de entrada y de salida
- Identificá al menos una guarda (por tiempo, por condición)
- Dibujá el diagrama completo (papel o Draw.io)
- Si es complejo, agrupá estados en uno compuesto

Entregable: diagrama en la carpeta del grupo.

# Autoevaluación · Ticket de Soporte

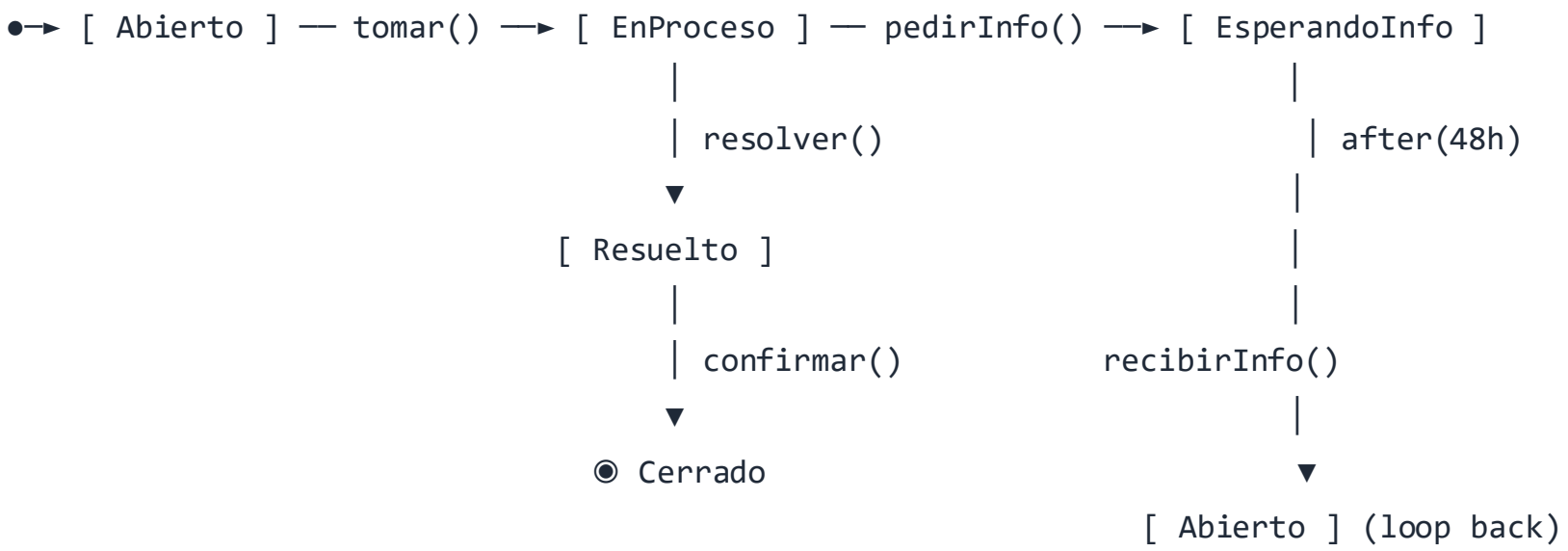
Enunciado

Un sistema de soporte recibe tickets. Al crearse queda Abierto. Un agente lo toma y pasa a EnProceso. Puede solicitar más información al usuario (EsperandoInfo). Si el usuario no responde en 48hs, vuelve a Abierto. Resuelto el problema, pasa a Resuelto y, si el usuario lo confirma, a Cerrado.

Consignas

- Listá los estados (al menos 5)
- Para cada estado, indicá evento de entrada y de salida
- Identificá al menos una guarda (por tiempo o por condición)
- Dibujá el diagrama de estados completo

# Autoevaluación · solución




Guardas clave: after(48h), [usuarioConfirma = true].

Estados: Abierto, EnProceso, EsperandoInfo, Resuelto, Cerrado.

# Conexión con clases anteriores y próximas

- Clase 14 (SSD) — los eventos del diagrama de estados son los mensajes del SSD que recibe el sistema
- Clase 13 (Dominio) — el diagrama de estados se hace para una clase del dominio con ciclo de vida relevante
- Clase 12 (UC) — las transiciones que llevan a estados finales suelen corresponder al éxito o fracaso de un caso de uso
- Clase 16 (Validación Triángulo Larman) — cruzar UC + Dominio + SSD + Estados como cierre

El diagrama de estados es la última pieza del bloque de análisis.

# Tarea para casa

- Para cada entidad clave del caso propio, evaluá si necesita diagrama de estados (criterio: ¿hay reglas de negocio temporales?)
- Construí los diagramas para las 2 entidades más relevantes
- Hacé la tabla de transiciones de cada una
- Validá la coherencia: cada operación del SSD debe corresponder a un evento del diagrama de estados (o ser una operación sin efecto sobre estados)
- Subí a la carpeta del grupo en ADO

Fecha: antes de Clase 16.

# Recursos

- Larman · Applying UML and Patterns (cap. 29 — State Machine Diagrams)
- Fowler · UML Distilled (cap. State Machine)
- Harel, D. (1987) · Statecharts: A Visual Formalism for Complex Systems
- OMG · UML 2.5.1 Specification §14 (State Machines)
- PlantUML · plantuml.com/state-diagram
- Draw.io · editor online

# Próxima clase · Clase 16

Validación del Triángulo de Larman — cierre del bloque de análisis.

Tenemos las cuatro vistas: UC + Dominio + SSD + Estados. Ahora vamos a cruzar las cuatro para validar que el análisis es consistente, completo y trazable, antes de pasar al diseño.

Es el último paso antes de transformar el análisis en clases de diseño con SOLID y patrones.

# Cierre · Preguntas

- ¿En su caso OAS, qué entidades tienen ciclo de vida relevante?
- ¿Cuándo conviene un estado compuesto y cuándo no?
- ¿Cómo se relaciona un evento de Estados con una operación del SSD?
- ¿Qué pasa si una transición tiene varias guardas que se solapan?

Dudas y consultas.